

StrayCare: A Community-Driven Platform for Animal Rescue and Adoption

BIB04 Sanath Bhat BIB21 Karttikey Chavali
BIB23 Hariom Khaladkar BIB26 Rajat Kulkarni

2025 – 2026

Contents

Acknowledgement	6
Abstract	7
Introduction to Project Topic	8
Overview	8
Motivation behind our Project	8
Aims And Objectives	8
Applying Software Engineering Approach	9
Literature Survey on “Rule-Based Approach in Natural Language Processing (NLP)”	10
Introduction	10
Previous Research	10
State of Art Algorithms	10
Handcrafted Rules in Rule-Based Sentiment & Severity Analysis	10
Software Requirements Specification	12
Introduction & Project Scope	12
System Features	12
External Interface Requirements	12
Non-Functional Requirements	12
Machine Learning Concepts and Implementation	13
Methodology and Algorithms	13
Confusion Matrix Concepts for Decision Tree Classifiers	13
Enhancements and Future Work	13
Software Testing and Quality Assurance	14

Introduction to Testing	14
Automated Testing	14
Manual Testing	14
System Design & Applications of Rule-Based NLP	16
System Architecture	16
Applications of Rule Based Approach in NLP systems	16
Methodology and Results	17
Technology Details Used in the Project	17
Procedure of the Project	17
Results	17
Conclusion	18
References	19
Project Plan 1.0	20

A PROJECT REPORT ON

“StrayCare: A Community-Driven Platform for Animal Rescue and Adoption”

SUBMITTED BY

BIB04 Sanath Bhat

BIB21 Karttikey Chavali

BIB23 Hariom Khaladkar

BIB26 Rajat Kulkarni

GUIDED BY

Dr. Bhavana Kanawade

**DEPARTMENT OF INFORMATION TECHNOLOGY
INTERNATIONAL INSTITUTE OF INFORMATION
TECHNOLOGY, PUNE - 411057**

SAVITRIBAI PHULE PUNE UNIVERSITY

2025 – 2026

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HINJEWADI, PUNE-57

DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate

This is to certify that the project report entitled “**StrayCare: A Community-Driven Platform for Animal Rescue and Adoption**” which is being submitted by:

- **BIB04 Sanath Bhat**
- **BIB21 Karttikey Chavali**
- **BIB23 Hariom Khaladkar**
- **BIB26 Rajat Kulkarni**

have partially completed the Project entitled “**StrayCare: A Community-Driven Platform for Animal Rescue and Adoption**”, under my guidance in partial fulfillment of the requirement for the award of the Bachelors of Engineering in Information Technology of International Institute of Information Technology, Hinjawadi, by Savitribai Phule Pune University for the academic year 2025 – 2026.

Dr. Bhavana Kanawade
Guide

Internal Examiner: _____

Dr. Jyoti Surve
Head of the Department

External Examiner: _____

Dr. Vaishali Patil
Principal

Date: _____

Acknowledgement

With immense pleasure, we are presenting this project report on “StrayCare: A Community Driven Platform for Animal Rescue and Adoption” as a part of the curriculum of B.E. Information Technology at International Institute of Information Technology, Hinjewadi, Pune.

It gives us the privilege to complete this report work under the valuable mentorship of Dr. Bhavana Kanawade. Her guidance, co-operation and encouragement has made a headway in the project. We are also extremely grateful to Dr. Vaishali Patil, Principal, and Dr. Jyoti Surve, Head of Department of Information Technology for providing all facilities and help for smooth progress of report work. We would also like to thank all the Staff Members of the Information Technology Department, Management, friends, and our family members, who have directly or indirectly guided and helped us for the preparation of this report and gave us unending support right from the stage the idea was conceived.

BIB04 Sanath Bhat

BIB21 Karttikey Chavali

BIB23 Hariom Khaladkar

BIB26 Rajat Kulkarni

Abstract

StrayCare is an innovative web-based platform developed to connect compassionate citizens with animal welfare organizations, creating a seamless channel for reporting and managing stray animal rescue cases. In many communities, people often witness injured or distressed animals but lack an easy and reliable way to request help. StrayCare fills this gap by offering a centralized system where citizens can quickly report incidents by uploading photos, sharing locations, and providing brief details about the situation.

To optimize the rescue workflow, StrayCare implements cutting-edge Artificial Intelligence. The platform utilizes a deterministic Rule-Based Natural Language Processing (NLP) algorithm for automated severity triaging based on user incident descriptions. Furthermore, it leverages Machine Learning through K-Means Clustering to identify accident hotspots and Tensorflow.js (MobileNet V2) for in-browser animal species recognition.

The platform empowers NGOs with a dedicated dashboard to receive, track, and respond to these rescue requests efficiently. Administrators maintain platform integrity through a control panel that verifies NGO authenticity. Built with a React frontend for an interactive user experience and a FastAPI backend for high-performance secure APIs, StrayCare ensures smooth communication. By bridging the gap between citizens and NGOs through algorithmic dispatch and transparent reporting, it transforms community empathy into real-world action—making it easier to save and protect the lives of stray animals in need.

Introduction to Project Topic

Overview

Across India's cities and towns, countless stray animals suffer from injuries, illness, and neglect in plain sight. While social media and local chat groups have become informal channels for reporting such incidents, these methods are fragmented, unreliable, and lack a structured response mechanism. This communication gap between the public and NGOs leads to delayed rescues. StrayCare is envisioned as a transformative digital solution to bridge this gap by offering a mobile-first, easy-to-use, location-aware platform augmented by intelligence.

Motivation behind our Project

Urban environments face an escalating crisis regarding stray animal welfare. When a citizen encounters an injured animal, finding an available NGO is highly fragmented. Existing solutions lack automated triaging and rely on manual phone calls. The motivation behind StrayCare is to inject AI-driven intelligence into this process, completely automating case severity assessment using Natural Language Processing (NLP), identifying high-risk geographic hotspots using clustering, and seamlessly dispatching the right rescue personnel.

Aims And Objectives

1. To develop a platform for real-time reporting of animal emergencies using GPS coordinates.
2. To implement a Rule-Based Natural Language Processing (NLP) algorithm for automated severity triaging based on user incident descriptions.
3. To deploy a Smart Dispatch algorithm utilizing the Haversine formula to optimize NGO allocation.
4. To integrate Machine Learning (K-Means Clustering) for identifying accident hotspots.
5. To execute in-browser machine learning (TensorFlow.js) for offline animal species recognition.

Applying Software Engineering Approach

StrayCare is developed using a structured software engineering methodology.

- **Architectural Design:** Three-tier architecture (Presentations, API Logic, DB).
- **Technology Stack:** React for the frontend and FastAPI (Python) for the backend.
- **Quality Assurance:** Comprehensive Automated and Manual testing strategy ensuring standard platform stability.
- **Agile Methodology:** Iterative development phases incorporating continuous AI capability upgrades.

Literature Survey on “Rule-Based Approach in Natural Language Processing (NLP)”

Introduction

Natural Language Processing (NLP) enables systems to understand and interpret human language. In emergency response platforms like StrayCare, understanding the urgency of a reported text is critical. Rule-based approaches in NLP rely on predefined linguistic rules and lexicons rather than statistical models, making them highly deterministic, fast, and reliable.

Previous Research

Historically, automated dispatch systems utilized simple keyword matching. Recent research in civic-tech has shifted towards hybrid models where rule-based systems function as a primary triage layer. Studies on urban emergency response indicate that rule-based NLP provides the lowest latency, which is the most critical metric in animal trauma cases where minutes determine life or death.

State of Art Algorithms

While Large Language Models (LLMs) like GPT-4 are current state-of-the-art, their reliance on external API calls introduces latency, privacy concerns, and recurring costs. For focused triage execution, handcrafted lexicons weighted by domain experts often outperform generalized probabilistic models in zero-shot environments.

Handcrafted Rules in Rule-Based Sentiment & Severity Analysis

In StrayCare, we implemented a custom Rule-Based NLP engine. The algorithm tokenizes the citizen’s case description and applies a weighted scoring system based on explicit predefined word arrays:

- **Critical (+100 pts):** Keywords like *bleeding*, *unconscious*, *hit by car*, *paralyzed*, *seizure*.

- **High (+60 pts):** Keywords like *injured, poisoned, trapped, bitten*.
- **Moderate (+30 pts):** Keywords like *limping, malnourished, abandoned*.

This deterministic approach allows the system to instantly assess severity and assign a priority label (Red/Orange/Yellow) to the case card, sorting it dynamically in the NGO's dashboard without relying on paid APIs.

Software Requirements Specification

Introduction & Project Scope

StrayCare creates a comprehensive digital ecosystem connecting citizens with animal rescue services. Users register and submit geo-tagged case reports with photographic evidence. Verified NGOs receive real-time notifications for new cases in their designated operational areas and manage their entire workflow through a dedicated dashboard.

System Features

- **User Authentication and Authorization:** Secure, role-based access for Citizens, NGOs, and Administrators via JSON Web Tokens (JWT).
- **Case Management & Workflow:** Citizens create cases (Pending status). NGOs accept/reject cases and post progress updates. Citizens track updates in real-time.
- **Pet Matchmaker:** Algorithmic lifestyle matching for adopting listed animals.

External Interface Requirements

- **User Interface:** Mobile-first, responsive Glassmorphism UI.
- **Software Interface:** SQLite Database managed through SQLAlchemy ORM.
- **Communication Interface:** HTTPS-secured RESTful APIs using standard JSON payloads.

Non-Functional Requirements

- **Performance:** Backend target API response under 500ms; frontend First Contentful Paint under 2 seconds.
- **Security:** Passwords irreversibly hashed via bcrypt. Role-based access control blocks unauthorized route access.
- **Scalability:** Stateless backend architecture easily horizontally scalable.

Machine Learning Concepts and Implementation

Methodology and Algorithms

StrayCare employs three distinct mathematical and ML mechanisms natively:

1. **K-Means Clustering (Unsupervised ML):** Used to power the “Zone Red Hotspot Map”. The algorithm processes the GPS coordinates (latitude/longitude) of all reported cases. It randomly initializes K centroids and iteratively assigns spatial points to the nearest centroid using Euclidean distance calculation.
2. **MobileNet V2 via TensorFlow.js (Supervised CNN):** Deployed entirely in the client’s browser. It classifies uploaded pet photos across predefined categories, functioning securely without cloud inference.
3. **Haversine Distance Formula for Smart Dispatch:** A geospatial algorithm used to calculate the great-circle distance between an emergency coordinate and localized NGOs.

Confusion Matrix Concepts for Decision Tree Classifiers

For supervised modules, model performance is evaluated using a Confusion Matrix containing True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Decision trees and CNNs alike depend on metrics such as Precision and Recall. The high precision guarantees that when StrayCare’s image recognition labels an animal as a “Dog” at 85% confidence, the NGO receives accurate contextual data.

Enhancements and Future Work

Future versions of the StrayCare ML pipeline will transition the rule-based NLP triaging to a fine-tuned lightweight transformer model. Additionally, time-series forecasting (ARIMA/Prophet) will be integrated with the K-Means clusters to predict future accident hotspots based on seasonal trends.

Software Testing and Quality Assurance

Introduction to Testing

To ensure the StrayCare platform is highly robust and capable of handling critical emergency data securely, a comprehensive testing strategy was formulated. This strategy combines Automated Testing for backend API stability and algorithmic accuracy, alongside rigorous Manual Testing encompassing end-to-end user workflows.

Automated Testing

Automated testing validates system integrity iteratively during CI/CD execution:

1. **Endpoint API Testing:** Automated requests were generated to test all endpoints (POST /report, GET /admin/hotspots, POST /users/login), ensuring strict JSON compliance and correct HTTP 200/401/404 response codes.
2. **Algorithmic Boundary Testing:** The Python algorithms, specifically the NLP rule-based triage and the Haversine distance calculator, were tested with edge-case coordinates and text arrays to prevent computational errors.
3. **Database Migration Tracking:** Evaluated the persistence of SQLite updates (like newly injected severity columns), preventing database schema mismatches during startup.

Manual Testing

Standardized manual execution verified actual Human-Computer Interaction (HCI):

1. **Role-Based Access Control (RBAC):** Testers actively attempted to breach routing controls (e.g., trying to load /admin/dashboard while logged in as a Citizen) to ensure the UI successfully repelled unauthorized access.
2. **End-to-End (E2E) Flow Execution:**
 - Citizen reports an injured dog (verifying browser GPS tracking).
 - AI accurately scores it as “Critical”.

- Testing admin account opens Smart Dispatch to evaluate Haversine proximity rankings.
 - NGO accepts the case and converts the resolved rescue into an Adoption Listing.
3. **Cross-Browser UI Validity:** Glassmorphism styling and TensorFlow.js memory utilization were evaluated manually across Chrome, Edge, and mobile Safari to guarantee universally fast loading.

System Design & Applications of Rule-Based NLP

System Architecture

The architecture is designed as a modern, multi-layered client-server system. It consists of a Presentation Layer (the React Frontend), a Logic Layer (the FastAPI Application), and a Data Layer (SQLite). The client-side (Browser) handles UI rendering and API requests via Axios. The server handles all logic, database models, and static `/upload` saving.

Applications of Rule Based Approach in NLP systems

The implementations within StrayCare demonstrate the immense value of rule-based NLP in real-world scenarios:

1. **Emergency Prioritization:** Instantly filtering high-trauma data points directly from unstructured community language.
2. **Automated Routing & Resource Allocation:** Using NLP data combined with geospatial proximity to dictate which NGO vehicle minimizes response latency.
3. **Medical Consistency:** Standardized handcrafted dictionaries guarantee no “hallucinations” (common in generative AI), allowing predictable, medical-grade triage parameters.

Methodology and Results

Technology Details Used in the Project

The frontend utilizes React.js (v18.x) for dynamic Single-Page Application behavior, CSS Modules for scoped styling, and Axios for HTTP requests. The backend leverages FastAPI for highly asynchronous REST API processing, Uvicorn as the ASGI server, and SQLite with SQLAlchemy for robust relational mapping. Authentication is fully backed by passlib (bcrypt) and python-jose (JWT).

Procedure of the Project

Development followed an iterative Agile methodology:

1. **Planning & Requirements:** Evaluated fragmented reporting infrastructures.
2. **Design:** API-first specifications and database modeling.
3. **Implementation:** Developed logic for citizen and NGO core loops, eventually injecting advanced math layers (Haversine, NLP, clustering).
4. **Testing & Refinement:** Passed through automated and manual E2E procedures.

Results

The system successfully coordinates high-speed case creation with instant dispatch allocation, providing transparent, real-time feedback loops between NGOs and citizens.

Conclusion

In addressing the complex challenge of stray animal welfare, the StrayCare project has culminated in the creation of a comprehensive digital ecosystem. By integrating intuitive reporting tools and mechanisms for accountability alongside advanced intelligence—ranging from NLP-based severity triaging to K-Means spatial clustering and in-browser image recognition—StrayCare provides a structured, technology-driven solution. The rigorous manual and automated testing protocols guarantee a secure, user-friendly, and bug-free experience. This platform ultimately transforms community empathy into optimized real-world operational execution, serving as a scalable blueprint for smart-city civic engagement.

References

1. FastAPI Documentation: <https://fastapi.tiangolo.com/>
2. TensorFlow.js Models & MobileNet: <https://www.tensorflow.org/js>
3. K-Means Clustering Algorithm Fundamentals & Spatial Mathematics.
4. Rule-Based Natural Language Processing Models and Lexicons.
5. React Framework Best Practices.
6. Jha et al., 2024 - “Using Machine Learning and AI to Find Homes for the Voiceless”
7. Rajwani et al., 2025 - “Enhancing Animal Rescue Efficiency Using AI for Real-Time Reporting”

Project Plan 1.0

Activity Planned / Module(s)		
Week	Covered	Status
Week 1	Requirement Gathering & Analysis – Define core modules.	Completed
Week 2	System Design & Database – Create ERD, DFDs, Use Case diagrams.	Completed
Week 3	Backend Core & Auth Module – FastAPI setup, JWT Security.	Completed
Week 4	Frontend Core & Auth Module – React routing and UI login.	Completed
Week 5	Case Reporting Module – POST /report, geolocation, Photo upload.	Completed
Week 6	NGO Case Management Module – NGO Dashboard and Data loops.	Completed
Week 7	Case Detail & Update Module – End-to-end Case timeline UI.	Completed
Week 8	AI Algorithms Module – Implemented rule-based NLP + Clustering.	Completed

	Activity Planned / Module(s)	
Week	Covered	Status
Week 9	Testing & QA – Executed Automated APIs + Manual E2E workflows.	Completed
Week 10	Security & Refinement – JWT hardening and UI mobile-responsiveness.	Completed
Week 11	Deployment Readiness – Server hosting preparation.	Completed
Week 12	Final Documentation – Completed Project Report integration.	Completed